

Partition DP Explained

Today 10:23 AM

Partition DP is used when a problem asks you to **split a range into smaller parts** and combine their answers.

General Idea

Suppose you have a range $[i \dots j]$.

Instead of solving it directly, **try every possible partition point k** .

```
[i .... k] | [k+1 .... j]
```

Solve both parts recursively and combine them.

Generic Formula

If $\text{cost}(i, j)$ is the answer for range $[i \dots j]$:

```
C++
dp[i][j] = min/max over all k

dp[i][j] =
    solve(i, k)
    + solve(k + 1, j)
    + cost(i, k, j);
```

where

```
k = i ... j-1
```

Recursive Template

```
C++
int solve(int i, int j){
    if(base case)
        return ...;

    if(dp[i][j] != -1)
        return dp[i][j];

    int ans = INF;
    for(int k = i; k < j; k++){
        int left = solve(i, k);
        int right = solve(k + 1, j);
        ans = min(ans, left + right + cost(i, k, j));
    }
    return dp[i][j] = ans;
}
```

How to Identify Partition DP

Ask yourself:

- ✓ Am I given an array/string/range?
- ✓ Do I need to split it into smaller segments?
- ✓ Can every split be independent?
- ✓ Am I taking minimum or maximum over all splits?

If **yes**, it's probably Partition DP.

Famous Problems

Problem	Partition
Matrix Chain Multiplication	Between matrices

Problem	Partition
Burst Balloons	Choose last balloon
Minimum Cost to Cut Stick	Choose next cut
Palindrome Partitioning II	Split string
Boolean Parenthesization	Split at operators
Scramble String	Split string

Example: Matrix Chain Multiplication

Matrices:

A B C D

Possible first partitions:

(A) (BCD)
(AB) (CD)
(ABC) (D)

Formula:

```
C++
solve(i, j) =
min(
    solve(i, k)
    + solve(k+1, j)
    + multiplication_cost
)
```

One-Line Memory Trick

"For every possible partition k , solve the left part, solve the right part, add the current cost, and take the minimum/maximum."

This is the core idea behind almost every Partition DP problem.

The **hardest part of MCM (Matrix Chain Multiplication)** is understanding **why the multiplication cost is:**

$arr[i-1] * arr[k] * arr[j]$

Let's make it very simple.

Step 1: Suppose we have matrices

A = 10 × 20
B = 20 × 30
C = 30 × 40

We store only their dimensions:

$arr = [10, 20, 30, 40]$

Meaning

A = $arr[0] \times arr[1] = 10 \times 20$
B = $arr[1] \times arr[2] = 20 \times 30$
C = $arr[2] \times arr[3] = 30 \times 40$

Step 2: How much does multiplying two matrices cost?

If

A = $p \times q$
B = $q \times r$

then

$$\text{Cost} = p \times q \times r$$

Example

$$\begin{aligned} &10 \times 20 \times 20 \times 30 \\ \text{Cost} \\ &= 10 \times 20 \times 30 \\ &= 6000 \text{ operations} \end{aligned}$$

Step 3: Suppose we split at $k = 1$

We have

(A) (BC)

Left side

$$\begin{aligned} &A \\ &10 \times 20 \end{aligned}$$

Right side

$$\begin{aligned} &BC \\ &20 \times 30 \times 30 \times 40 \\ \text{Cost} \\ &20 \times 30 \times 40 \end{aligned}$$

After multiplying BC

Result becomes

$$20 \times 40$$

Now multiply

$$\begin{aligned} &10 \times 20 \\ &\text{with} \\ &20 \times 40 \end{aligned}$$

Cost

$$10 \times 20 \times 40$$

Total

$$\begin{aligned} &\text{cost}(A) \\ &+ \\ &\text{cost}(BC) \\ &+ \\ &10 \times 20 \times 40 \end{aligned}$$

Step 4: Where does $\text{arr}[i-1] * \text{arr}[k] * \text{arr}[j]$ come from?

Suppose

$\text{solve}(i, j)$

splits at k

$(i \dots k)$
 $(k+1 \dots j)$

Left matrix becomes

$$\text{arr}[i-1] \times \text{arr}[k]$$

Right matrix becomes

```
arr[k] * arr[j]
```

Now multiply these two matrices.

Cost

```
(arr[i-1])
×
(arr[k])
×
(arr[j])
```

That's why

```
C++
cost = arr[i-1] * arr[k] * arr[j];
```

Example

```
arr = [10,20,30,40]
solve(1,3)
Matrices
A B C
```

Split at

```
k=2
(AB) (C)
```

Left

```
AB
10×20 × 20×30
↓
10×30
```

Right

```
C
30×40
```

Now multiply

```
10×30
×
30×40
```

Cost

```
10×30×40
=12000
```

Formula

```
arr[i-1] * arr[k] * arr[j]
=arr[0] * arr[2] * arr[3]
=10*30*40
```

Exactly the same.

The trick to always remember

For every partition:

$(i \dots k) \mid (k+1 \dots j)$

1. Solve the left part.
2. Solve the right part.
3. **The left result matrix will always have dimensions $arr[i-1] \times arr[k]$.**
4. **The right result matrix will always have dimensions $arr[k] \times arr[j]$.**
5. Multiply those two resulting matrices.

So the transition is always:

```
C++
dp[i][j] = min(
    solve(i, k)
    + solve(k + 1, j)
    + arr[i - 1] * arr[k] * arr[j]
);
```

Once you understand **what the dimensions of the left and right result matrices are**, the formula becomes natural instead of something to memorize.

Let's build the recursion tree **step by step**. This is the best way to understand how MCM works.

Given

Dimensions:

```
arr = [10, 20, 30, 40, 50]
```

Matrices:

```
A = 10x20
B = 20x30
C = 30x40
D = 40x50
```

We call

```
C++
solve(1, 4)
```

because matrices are numbered **1 to 4**.

Level 0

We try **every possible split (k)**.

```
solve(1, 4)
      /   ABCD   \
     k=1  k=2    k=3
```

which means

```
(A) (BCD)
(AB) (CD)
(ABC) (D)
```

Now recursion starts.

Split k = 1

```
solve(1, 4)
├── solve(1, 1)    // A
└── solve(2, 4)    // BCD
```

Now `solve(2, 4)` again tries every split.

```
solve(2, 4)
      BCD
```

$\begin{matrix} / \\ k=2 \end{matrix}$
 $\begin{matrix} \backslash \\ k=3 \end{matrix}$

(B) (CD)

(BC) (D)

So

```

solve(2,4)
├── solve(2,2)
└── solve(3,4)

```

and

```

solve(3,4)
  CD
  |
  k=3

```

```

solve(3,4)
├── solve(3,3)
└── solve(4,4)

```

These are single matrices, so recursion stops.

Return from Bottom

First return

```

solve(3,3)=0
solve(4,4)=0

```

Now calculate

```

solve(3,4)
cost =
0
+
0
+
30×40×50
=60000

```

Return

```

solve(3,4)=60000

```

Now compute

```

solve(2,4)
Split k=2
cost
=
solve(2,2)
+
solve(3,4)
+
20×30×50
=
0
+
60000
+
30000
=
90000

```

Now second partition inside `solve(2,4)`:

(BC) (D)

Need

```
solve(2,3)
```

```
solve(2,3)
```

```
B C
```

```
solve(2,3)
```

```
├─ solve(2,2)
└─ solve(3,3)
```

returns

```
20×30×40
```

```
=
24000
```

Now

```
solve(2,4)
```

```
k=3
```

```
=
24000
+
0
+
20×40×50
```

```
=
24000
+
40000
=
64000
```

Take minimum

```
min(90000, 64000)
```

```
=
64000
```

So

```
solve(2,4)=64000
```

Now compute the top call for **k=1**

```
solve(1,4)
```

```
=
solve(1,1)
+
solve(2,4)
+
10×20×50
```

```
=
0
+
64000
+
10000
=
74000
```

This is **only one partition (k=1)**.

Now root tries k = 2

```
(AB) (CD)
```

```
solve(1,2)
```

```
A B
```

returns

```
10×20×30
```

```
=
6000
```

Already know

```
solve(3,4)
=
60000
```

So

```
6000
+
60000
+
10×30×50
=
6000
+
60000
+
15000
=
81000
```

Root tries k = 3

```
(ABC) (D)
```

Need

```
solve(1,3)
```

which again tries

```
(A) (BC)
(AB) (C)
```

Eventually

```
solve(1,3)=18000
```

Now

```
18000
+
0
+
10×40×50
=
18000
+
20000
=
38000
```

Finally root compares

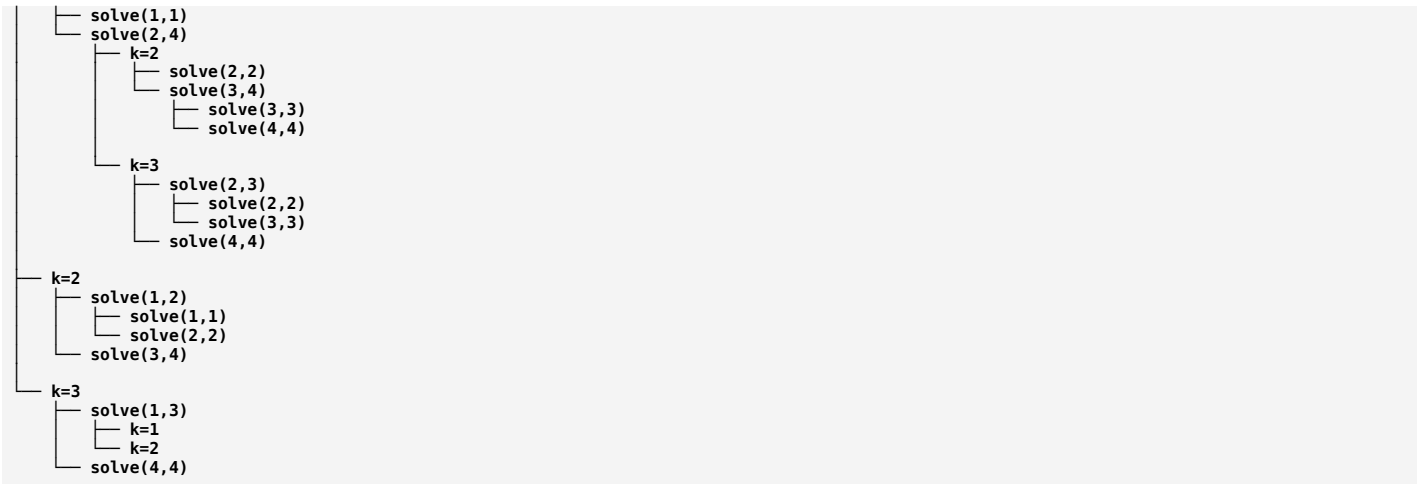
```
k=1
74000
k=2
81000
k=3
38000
```

Answer

```
38000
```

Complete Recursion Tree

```
solve(1,4)
├── k=1
```



The key idea to remember

- **Going down the tree:** Each node tries **every possible split (k)**, recursively solving the left and right subproblems.
- **Coming back up:** Each node computes:

```
C++
cost = leftCost + rightCost + arr[i-1] * arr[k] * arr[j];
```

for **every** split k , and returns the **minimum** cost to its parent.

This is exactly how Partition DP works: **split** → **recurse** → **return costs** → **choose the best split**.

The **biggest trick** in converting **recursion to tabulation** is to ask:

"Which smaller states does my current state depend on?"

Let's use the same MCM example.

Step 1: Recursive function

```
C++
solve(i, j)
```

means

Minimum cost to multiply matrices from i to j .

Transition:

```
C++
for(int k=i;k<j;k++)
{
    ans=min(ans,
        solve(i,k)
        + solve(k+1,j)
        + arr[i-1]*arr[k]*arr[j]);
}
```

Notice the dependencies.

To compute

```
dp[1][4]
```

we need

```
dp[1][1]
dp[2][4]

dp[1][2]
dp[3][4]

dp[1][3]
dp[4][4]
```

Those are **smaller ranges**.

Step 2: Draw the DP table

Suppose

`n = 4 matrices`

DP table

		j			
		1	2	3	4
i	1	dp11	dp12	dp13	dp14
	2		dp22	dp23	dp24
	3			dp33	dp34
	4				dp44

Only the **upper triangle** is used because $i \leq j$.

Step 3: Fill the base case

A single matrix needs **no multiplication**.

```
dp[1][1]=0
dp[2][2]=0
dp[3][3]=0
dp[4][4]=0
```

Table becomes

	1	2	3	4
1	0			
2		0		
3			0	
4				0

Step 4: Which cell comes next?

Look at recursion.

To compute

`dp[1][2]`

Need

```
dp[1][1]
dp[2][2]
```

Already computed.

Similarly

`dp[2][3]`

Need

```
dp[2][2]
dp[3][3]
```

Already computed.

Same for

`dp[3][4]`

So after diagonals, compute **length = 2**.

	1	2	3	4
1	0	✓		
2		0	✓	
3			0	✓

4 0

Step 5: Next length = 3

Now compute

`dp[1][3]`

Needs

`dp[1][1]`
`dp[2][3]`

`dp[1][2]`
`dp[3][3]`

Everything already exists.

Similarly

`dp[2][4]`

Depends on

`dp[2][2]`
`dp[3][4]`

`dp[2][3]`
`dp[4][4]`

Already computed.

	1	2	3	4
1	0	✓	✓	
2		0	✓	✓
3			0	✓
4				0

Step 6: Finally length = 4

Now compute

`dp[1][4]`

Needs

`dp[1][1]`
`dp[2][4]`

`dp[1][2]`
`dp[3][4]`

`dp[1][3]`
`dp[4][4]`

All already computed.

	1	2	3	4
1	0	✓	✓	✓
2		0	✓	✓
3			0	✓
4				0

Done.

Filling Order

This is the **most important thing** to remember:

Diagonal

↓

Length = 2

```
↓
Length = 3
↓
Length = 4
↓
...
↓
Length = n
```

Not row-wise.

Not column-wise.

Always by **increasing range length**.

Code

```
C++
for(int i = 1; i <= n; i++)
    dp[i][i] = 0;

for(int len = 2; len <= n; len++)
{
    for(int i = 1; i <= n - len + 1; i++)
    {
        int j = i + len - 1;
        dp[i][j] = INT_MAX;

        for(int k = i; k < j; k++)
        {
            dp[i][j] = min(dp[i][j],
                           dp[i][k] +
                           dp[k+1][j] +
                           arr[i-1] * arr[k] * arr[j]);
        }
    }
}
```

Memory Trick for Partition DP

Whenever your recursive state is:

```
C++
solve(i, j)
```

ask yourself:

- **Base case:** $i == j$
- **Smaller states:** shorter ranges
- **So in tabulation:** fill by **range length**, from shortest to longest.

This pattern works not only for **Matrix Chain Multiplication**, but also for **Burst Balloons**, **Minimum Cost to Cut a Stick**, **Boolean Parenthesization**, **Palindrome Partitioning**, and most other Partition DP problems.

Let's understand it **once and forever**. Most students get confused by the `len <= n` loop, so let's derive it instead of memorizing it.

Problem

Dimensions:

```
arr = [10,20,30,40,50]
```

Matrices

```
A = 10×20
B = 20×30
C = 30×40
D = 40×50
```

Number of matrices

```
n = arr.size() - 1 = 4
```

So our DP table is

	1	2	3	4
1	dp11	dp12	dp13	dp14
2		dp22	dp23	dp24
3			dp33	dp34
4				dp44

Complete Code

```

C++
#include <bits/stdc++.h>
using namespace std;

int matrixMultiplication(vector<int> &arr) {
    int n = arr.size() - 1;    // number of matrices
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
    // length of matrix chain
    for(int len = 2; len <= n; len++){
        for(int i = 1; i <= n - len + 1; i++){
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for(int k = i; k < j; k++){
                int cost =
                    dp[i][k]
                    + dp[k + 1][j]
                    + arr[i - 1] * arr[k] * arr[j];
                dp[i][j] = min(dp[i][j], cost);
            }
        }
    }
    return dp[1][n];
}

int main(){
    vector<int> arr = {10,20,30,40,50};
    cout << matrixMultiplication(arr);
}

```

Output

```
38000
```

Dry Run

Initially

	1	2	3	4
1	0	-	-	-
2		0	-	-
3			0	-
4				0

Diagonal is 0 because one matrix requires no multiplication.

Why does length start from 2?

Length means

```
number of matrices
```

Length 1

```

A
already known
cost = 0

```

So we don't compute it.

First unknown chain is

```
AB
```

Length = 2

Iteration 1

```
len = 2
```

i = 1

```
j = i + len - 1
= 1 + 2 - 1
= 2
```

We calculate

```
dp[1][2]
```

Only one split

```
A | B
```

Cost

```
0
+
0
+
10×20×30
=
6000
```

```
dp[1][2]=6000
```

i=2

```
j=3
```

Compute

```
dp[2][3]
```

```
B | C
```

Cost

```
20×30×40
=
24000
```

i=3

```
j=4
```

Compute

```
dp[3][4]
```

```
C | D
```

Cost

```
30×40×50
=
60000
```

Table

	1	2	3	4
1	0	6000		
2		0	24000	
3			0	60000
4				0

Iteration 2

len=3

Now chain has

3 matrices

i=1

j=3

Need

dp[1][3]

Possible splits

k=1

A | BC

0
+
24000
+
10×20×40
=
32000

k=2

AB | C

6000
+
0
+
10×30×40
=
18000

Take minimum

18000

dp[1][3]=18000

i=2

j=4

Need

dp[2][4]

Split 1

B | CD

```
0
+
60000
+
20x30x50
=
90000
```

Split 2

```
BC | D
```

```
24000
+
0
+
20x40x50
=
64000
```

Take minimum

```
64000
```

Table

	1	2	3	4
1	0	6000	18000	
2		0	24000	64000
3			0	60000
4				0

Iteration 3

```
len=4
```

Entire chain

```
ABCD
```

Only

```
i=1
j=4
```

Need

```
dp[1][4]
```

k=1

```
A | BCD
0
+
64000
+
10x20x50
=
74000
```

k=2

```
AB | CD
```

```
6000
+
60000
+
10×30×50
=
81000
```

k=3

```
ABC | D
18000
+
0
+
10×40×50
=
38000
```

Take minimum

```
38000
```

Answer

```
dp[1][4]=38000
```

Why does len go till <= n?

This is the most confusing part.

We have

```
A B C D
```

Number of matrices

```
4
```

Possible chain lengths are

```
Length=1
A
B
C
D
```

Already known.

```
Length=2
AB
BC
CD
```

Need to compute.

```
Length=3
ABC
BCD
```

Need to compute.

```
Length=4
ABCD
```

Need to compute.

There is **no chain of length 5** because only **4 matrices exist**.

Therefore

```
C++
for(int len=2; len<=n; len++)
```

When $n = 4$, the loop runs:

```
len = 2
len = 3
len = 4
```

Exactly the lengths that need computation.

Why $i \leq n - len + 1$?

Suppose

```
len = 3
```

Possible chains are

```
ABC
```

```
BCD
```

Starting indices are

```
1
2
```

Maximum starting index is

```
2
```

Formula gives

```
n-len+1
=
4-3+1
=
2
```

Perfect.

If $i = 3$

```
Chain
CDE
```

doesn't exist.

That's why the loop stops at

```
C++
i <= n - len + 1;
```

Easy way to remember

Think of len as "how many matrices are in the current chain?"

- $len = 1 \rightarrow$ base case (already 0)
- $len = 2 \rightarrow$ chains of 2 matrices
- $len = 3 \rightarrow$ chains of 3 matrices
- ...
- $len = n \rightarrow$ the entire chain (the final answer)

So `len` naturally goes from 2 to `n`.

Today 3:18 PM

Yes. There are **two common ways** to write tabulation for MCM.

Method 1 (Most Common) - By Chain Length

```
C++
for (int len = 2; len <= n; len++)
    for (int i = 1; i <= n - len + 1; i++)
```

This fills the DP table **diagonal by diagonal**.

Method 2 (Striver Style) - Start from Last Index

This fills the DP table **from bottom to top**.

Code

```
C++
#include <bits/stdc++.h>
using namespace std;

int matrixMultiplication(vector<int> &arr) {
    int n = arr.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));

    // i goes backwards
    for (int i = n - 1; i >= 1; i--) {
        // j goes forwards
        for (int j = i + 1; j <= n - 1; j++) {
            dp[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int cost =
                    dp[i][k]
                    + dp[k + 1][j]
                    + arr[i - 1] * arr[k] * arr[j];

                dp[i][j] = min(dp[i][j], cost);
            }
        }
    }

    return dp[1][n - 1];
}

int main() {
    vector<int> arr = {10, 20, 30, 40, 50};
    cout << matrixMultiplication(arr);

    return 0;
}
```

Why does this work?

Suppose

```
arr = [10 20 30 40 50]
```

Matrices

```
A B C D
```

The DP table is

	j			
	1	2	3	4
1	-	-	-	-
2		-	-	-
3			-	-
4				-

Initially every diagonal is

```
dp[i][i]=0
```

First iteration

```
i = 4
```

Inner loop

```
j = 5
```

Doesn't run.

Nothing happens.

Second iteration

```
i = 3
```

```
j = 4
```

We calculate

```
dp[3][4]
```

which depends on

```
dp[3][3]
```

```
dp[4][4]
```

Already computed (both are 0).

Table becomes

	1	2	3	4
1				
2				
3			0	✓
4				0

Third iteration

```
i=2
```

First

```
j=3
```

Need

```
dp[2][2]
```

```
dp[3][3]
```

Already available.

Compute

```
dp[2][3]
```

Next

```
j=4
```

Need

```
dp[2][2]
```

```
dp[3][4]
dp[2][3]
dp[4][4]
```

Notice

```
dp[3][4]
```

was computed when

```
i=3
```

and

```
dp[2][3]
```

was computed just now.

So everything is available.

Table

	1	2	3	4
1				
2		0	✓	✓
3			0	✓
4				0

Fourth iteration

```
i=1
```

```
j=2
```

Compute

```
dp[1][2]
```

```
j=3
```

Need

```
dp[1][2]
dp[2][3]
```

Already available.

```
j=4
```

Need

```
dp[1][2]
dp[3][4]
dp[1][3]
dp[2][4]
```

Everything is already computed.

Finally

	1	2	3	4
1	0	✓	✓	✓
2		0	✓	✓
3			0	✓
4				0

Why do we go from the last row?

Look at the recurrence:

```
C++
dp[i][j]
```

depends on

```
C++
dp[i][k]
```

and

```
C++
dp[k+1][j]
```

Notice one dependency is

```
k+1
```

which means a **larger row index**.

For example,

```
dp[2][4]
```

needs

```
dp[3][4]
```

Therefore,

```
Row 3
```

must already be computed before

```
Row 2
```

That's exactly why we iterate

```
C++
for(int i=n-1; i>=1; i--)
```

from bottom to top.

Difference between the two methods

Method 1 (Chain Length)	Method 2 (Reverse i)
Fill by diagonals (chain length 2, 3, 4...)	Fill from bottom row to top row
Easy to understand because it follows increasing subproblem size	Matches the dependency order directly
Uses len, i, j	Uses only i, j
Preferred in textbooks	Common in Striver's DP series and competitive programming

The most important thing

Both methods compute the **same cells in a different order**.

Method 1 (chain length):

```
dp[1][2]
dp[2][3]
dp[3][4]

↓

dp[1][3]
dp[2][4]

↓

dp[1][4]
```

Method 2 (reverse i):

```
dp[3][4]
```

```
↓
```

```
dp[2][3]  
dp[2][4]
```

```
↓
```

```
dp[1][2]  
dp[1][3]  
dp[1][4]
```

Although the order is different, **every state is computed only after all the states it depends on are already available**, so both produce the same final answer.

ChatGPT can make mistakes. Check important info.